

DEVOPS PRACTICA

Nombre: Cristofer Sntaxi

Curso: 5to C

Contenido

1. Investigación.....	3
1.1 Control de Versiones	3
1.2 Git	3
1.3 GitHub.....	4
1.4 Repositorio Local.....	4
1.5 Repositorio Remoto	5
1.6 Commit	5
1.7 Branch (Rama).....	5
1.8 Merge	6
1.9 Pull Request	6
1.10 Fork.....	6
1.11 Conflictos de Integración	7
1.12 Integración Continua (CI).....	7
1.13 Entrega Continua (Continuous Delivery)	8
1.14 Despliegue Continuo (Continuous Deployment).....	8
1.15 GitHub Actions	8
2. Demostración Práctica.....	9
Paso 1: Creación del repositorio en GitHub	9
Paso 2: Creación de ramas de trabajo por cada integrante.....	10
Paso 3 y 4 : Commits realizados por ambos integrantes y Publicación de cambios al repositorio remoto.....	11
Paso 5: Creación de Pull Requests.....	12
Paso 6: Revisión de código	13
Paso 7: Aprobación de Pull Requests.....	13

Paso 8: Ejecución del proceso de Merge	14
Paso 9: Aprobar Pull Request	15
Paso 10: Generación y resolución de conflictos de integración	16
Paso 10: Ejecución exitosa del workflow de GitHub Actions.	17
3. Información Adicional.....	17
Datos del Estudiante	17
Conclusiones	17
Bibliografía	18

1. Investigación

A continuación se presenta la investigación documentada de los conceptos fundamentales relacionados con el control de versiones, Git, GitHub y las prácticas de integración y despliegue continuo.

1.1 Control de Versiones

Definición: Sistema que registra cambios realizados sobre un archivo o conjunto de archivos a lo largo del tiempo, de modo que se puedan recuperar versiones específicas posteriormente.

Utilidad: Permite rastrear la historia de modificaciones, revertir a estados anteriores, comparar cambios entre versiones y trabajar colaborativamente sin riesgo de sobrescribir el trabajo de otros.

Ejemplo práctico: Un equipo de desarrollo trabaja en un proyecto web. Sin control de versiones, si dos programadores editan el mismo archivo, uno perdería los cambios del otro. Con control de versiones, ambos pueden trabajar en paralelo y fusionar sus cambios de manera controlada.

Fuente: Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. <https://git-scm.com/book/en/v2>

1.2 Git

Definición: Sistema de control de versiones distribuido de código abierto, diseñado para manejar desde proyectos pequeños hasta proyectos muy grandes con velocidad y eficiencia. Fue creado por Linus Torvalds en 2005.

Utilidad: Git permite a múltiples desarrolladores trabajar en el mismo proyecto simultáneamente, manteniendo un historial completo de cambios, ramificaciones para desarrollo paralelo, y capacidades de fusión y resolución de conflictos.

Ejemplo práctico: Inicializar un repositorio Git en un proyecto local: `git init`. Clonar un repositorio remoto: `git clone https://github.com/usuario/repo.git`. Ver el estado de los archivos: `git status`.

Fuente: Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. <https://git-scm.com/book/en/v2>

1.3 GitHub

Definición: Plataforma de desarrollo colaborativo que utiliza Git como sistema de control de versiones. Permite alojar repositorios en la nube, gestionar proyectos y facilitar la colaboración entre desarrolladores. Fue fundada en 2008 y adquirida por Microsoft en 2018.

Utilidad: Proporciona una interfaz web para gestionar repositorios Git, herramientas de seguimiento de issues, Pull Requests, Actions para CI/CD, GitHub Pages para hosting estático, y un marketplace de integraciones.

Ejemplo práctico: Crear un nuevo repositorio en GitHub, configurar un README.md, agregar colaboradores desde Settings > Manage access, y utilizar GitHub Actions para automatizar pruebas cada vez que se hace push a main.

Fuente: GitHub Docs. (2024). Getting started with GitHub. <https://docs.github.com/en/get-started>

1.4 Repositorio Local

Definición: Copia del repositorio de código que reside en la máquina del desarrollador. Contiene la historia completa del proyecto, todas las ramas, tags y configuraciones.

Utilidad: Permite trabajar sin conexión a internet, realizar commits locales rápidamente, experimentar con ramas y cambios sin afectar al repositorio remoto hasta que se decida sincronizar.

Ejemplo práctico: Ejecutar git init en una carpeta de proyecto crea un repositorio local con la carpeta .git que almacena toda la historia del proyecto. Con git log se puede ver el historial de commits locales.

Fuente: Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. <https://git-scm.com/book/en/v2>

1.5 Repositorio Remoto

Definición: Versión del repositorio alojada en un servidor externo (como GitHub, GitLab o Bitbucket). Actúa como punto central de colaboración para todos los miembros del equipo.

Utilidad: Sirve como respaldo del código, punto de sincronización para equipos distribuidos, y plataforma para integraciones con herramientas de CI/CD, revisión de código y despliegue.

Ejemplo práctico: Conectar un repositorio local a uno remoto: `git remote add origin https://github.com/usuario/repo.git`. Enviar cambios: `git push origin main`. Descargar cambios: `git pull origin main`.

Fuente: Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. <https://git-scm.com/book/en/v2>

1.6 Commit

Definición: Instantánea de los cambios realizados en el repositorio en un momento específico. Cada commit tiene un hash SHA-1 único que lo identifica, un mensaje descriptivo, autor y fecha.

Utilidad: Permite guardar el estado del proyecto en puntos específicos, crear un historial de cambios organizado, y facilita la navegación entre diferentes versiones del código.

Ejemplo práctico: Después de modificar archivos: `git add archivo.txt` (agregar al área de staging), luego `git commit -m "Agrega funcionalidad de login de usuarios"`. Buena práctica: mensajes descriptivos en imperativo.

Fuente: Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. <https://git-scm.com/book/en/v2>

1.7 Branch (Rama)

Definición: Línea independiente de desarrollo que permite trabajar en características, correcciones o experimentos sin afectar la rama principal (generalmente main o master).

Utilidad: Facilita el desarrollo paralelo, aislamiento de funcionalidades, experimentación segura, y organización del trabajo en equipo mediante estrategias como Git Flow o GitHub Flow.

Ejemplo práctico: Crear una nueva rama: `git branch feature-login`. Cambiar a ella: `git checkout feature-login` (o `git switch -c feature-login`). Ver ramas existentes: `git branch -a`.

Fuente: Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. <https://git-scm.com/book/en/v2>

1.8 Merge

Definición: Operación de Git que combina los cambios de una rama en otra. Une las historias de desarrollo, integrando el trabajo realizado en paralelo.

Utilidad: Permite integrar funcionalidades completadas al código principal, consolidar el trabajo de múltiples ramas de desarrollo, y mantener un flujo de trabajo organizado en equipos.

Ejemplo práctico: Para fusionar feature-login en main: `git checkout main`, luego `git merge feature-login`. Si no hay conflictos, Git crea un commit de merge automáticamente.

Fuente: Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. <https://git-scm.com/book/en/v2>

1.9 Pull Request

Definición: Mecanismo en GitHub para solicitar la revisión e integración de cambios de una rama a otra. Permite la discusión, revisión de código y aprobación antes del merge.

Utilidad: Facilita la revisión de código por pares, discusión de cambios, validación automática mediante CI/CD, y documentación de decisiones técnicas antes de integrar al código principal.

Ejemplo práctico: Después de push de feature-login, en GitHub aparece un botón "Compare & pull request". Se describe el cambio, se asignan revisores, y tras aprobación se hace merge.

Fuente: GitHub Docs. (2024). About pull requests. <https://docs.github.com/en/pull-requests>

1.10 Fork

Definición: Copia completa de un repositorio en la cuenta personal de GitHub. Permite experimentar libremente con cambios sin afectar el proyecto original.

Utilidad:Fundamental para contribuir a proyectos de código abierto, proponer mejoras a repositorios donde no se tiene acceso de escritura, y personalizar proyectos existentes para necesidades específicas.

Ejemplo práctico:Hacer fork del repositorio facebook/react, clonarlo localmente, realizar cambios, push a tu fork, y crear un Pull Request al repositorio original proponiendo la mejora.

Fuente: GitHub Docs. (2024). Working with forks. <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/working-with-forks>

1.11 Conflictos de Integración

Definición:Situación que ocurre cuando Git no puede fusionar automáticamente cambios contradictorios en el mismo archivo. Requiere intervención manual para resolver.

Utilidad:Aunque parecen negativos, los conflictos son un mecanismo de seguridad que obliga a revisar cambios contradictorios, evitando sobrescritura accidental de código importante.

Ejemplo práctico:Si dos desarrolladores modifican la misma línea de un archivo, Git marca el conflicto con <<<<<<< HEAD, =====, >>>>>>> rama. Se edita el archivo, se eliminan los marcadores, se hace add y commit.

Fuente: Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. <https://git-scm.com/book/en/v2>

1.12 Integración Continua (CI)

Definición:Práctica de desarrollo donde los desarrolladores integran código a un repositorio compartido frecuentemente, validado con builds y pruebas automatizadas.

Utilidad:Detecta errores temprano, automatiza pruebas de regresión, garantiza que el código siempre esté en estado desplegable, y reduce el riesgo de integración al final de un ciclo de desarrollo.

Ejemplo práctico:Cada push a main ejecuta automáticamente: compilación del proyecto, ejecución de tests unitarios, análisis de código estático, y reporte de cobertura de pruebas.

*Fuente: Fowler, M. (2006). Continuous Integration.
<https://martinfowler.com/articles/continuousIntegration.html>*

1.13 Entrega Continua (Continuous Delivery)

Definición: Extensión de CI donde el código integrado se prepara automáticamente para su liberación a producción. El despliegue final requiere aprobación manual.

Utilidad: Mantiene el software siempre en estado listo para producción, reduce el tiempo entre desarrollo y liberación, y minimiza el riesgo asociado a los despliegues.

Ejemplo práctico: Tras pasar todas las pruebas CI, se genera un artefacto de despliegue (Docker image, .jar, etc.) listo para producción. Un botón manual en el dashboard permite el despliegue.

Fuente: Humble, J., & Farley, D. (2010). Continuous Delivery. Addison-Wesley.

1.14 Despliegue Continuo (Continuous Deployment)

Definición: Práctica donde los cambios que pasan todas las etapas del pipeline se despliegan a producción automáticamente, sin intervención manual.

Utilidad: Acelera drásticamente el ciclo de feedback, permite liberar múltiples veces al día, y requiere un sistema de monitoreo robusto para detectar y revertir problemas rápidamente.

Ejemplo práctico: Netflix y Etsy despliegan cientos de veces al día. Cada commit que pasa pruebas automatizadas se lanza a producción con técnicas de despliegue canario.

Fuente: Humble, J., & Farley, D. (2010). Continuous Delivery. Addison-Wesley.

1.15 GitHub Actions

Definición: Plataforma de automatización integrada en GitHub que permite crear workflows de CI/CD directamente en el repositorio mediante archivos YAML.

Utilidad: Automatiza pruebas, builds, despliegues, y cualquier tarea de desarrollo. Soporta ejecución en múltiples sistemas operativos, matrices de pruebas, y un marketplace de acciones reutilizables.

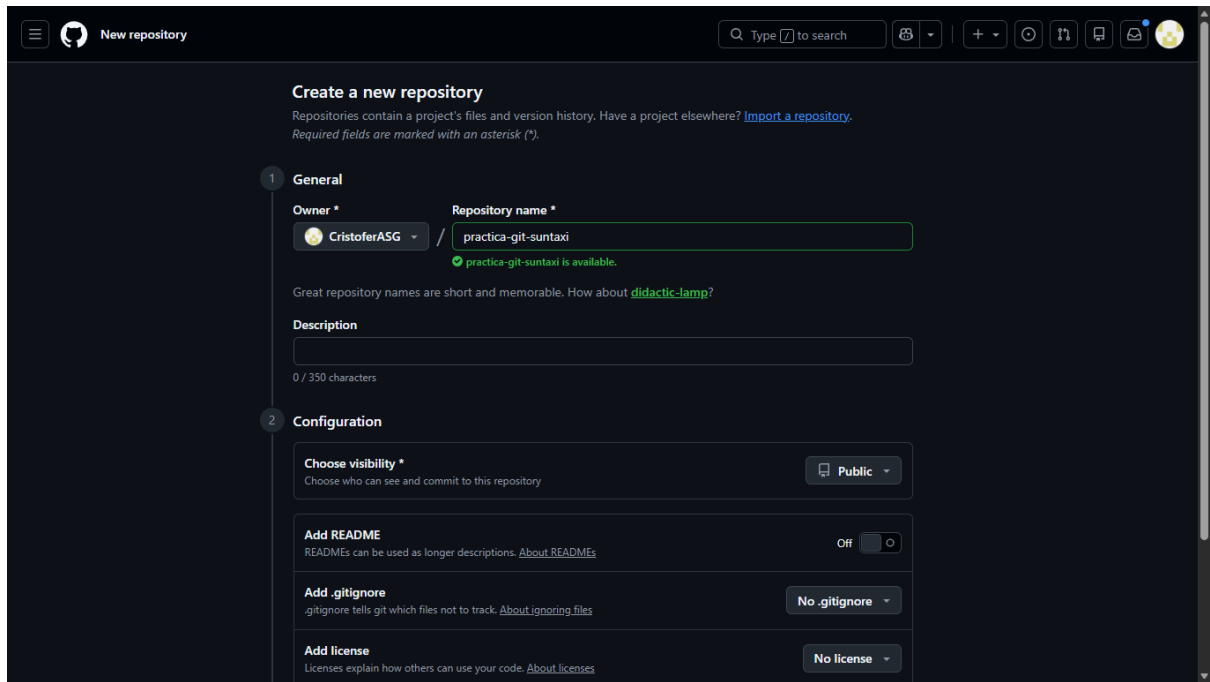
Ejemplo práctico: Un workflow `.github/workflows/ci.yml` que se ejecuta en push a main, configura Python, instala dependencias, ejecuta tests con pytest, y construye/pushea una imagen Docker a GitHub Container Registry.

Fuente: GitHub Docs. (2024). GitHub Actions. <https://docs.github.com/en/actions>

2. Demostración Práctica

A continuación se documenta paso a paso la práctica realizada, incluyendo los comandos ejecutados y espacios para las capturas de pantalla correspondientes.

Paso 1: Creación del repositorio en GitHub



Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * CristoferASG / Repository name * practica-git-suntaxi
✔ practica-git-suntaxi is available.

Great repository names are short and memorable. How about [didactic-lamp](#)?

Description
0 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository Public

Add README
READMEs can be used as longer descriptions. [About READMEs](#) Off

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#) No .gitignore

Add license
Licenses explain how others can use your code. [About licenses](#) No license

Paso 2: Creación de ramas de trabajo por cada integrante

```
C:\Users\Usuario\Documents\5to\Devops\Conexion>git checkout -b estudiante-a
Switched to a new branch 'estudiante-a'

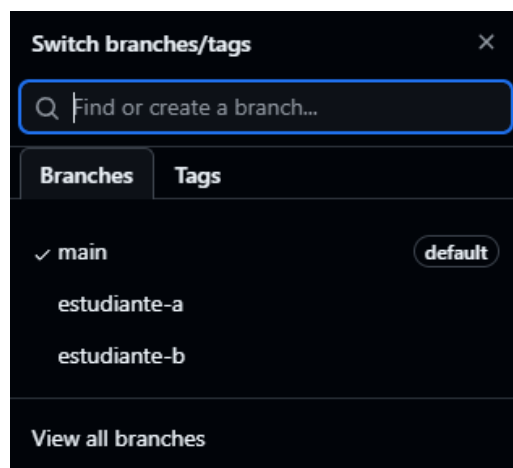
C:\Users\Usuario\Documents\5to\Devops\Conexion>git push -u origin estudiante-a
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'estudiante-a' on GitHub by visiting:
remote:   https://github.com/CristoferASG/practica-git-suntaxi/pull/new/estudiante-a
remote:
To https://github.com/CristoferASG/practica-git-suntaxi.git
 * [new branch]      estudiante-a -> estudiante-a
branch 'estudiante-a' set up to track 'origin/estudiante-a'.

C:\Users\Usuario\Documents\5to\Devops\Conexion>git checkout main
M      .dockerignore
M      .gitignore
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

C:\Users\Usuario\Documents\5to\Devops\Conexion>git checkout -b estudiante-b
Switched to a new branch 'estudiante-b'

C:\Users\Usuario\Documents\5to\Devops\Conexion>git push -u origin estudiante-b
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'estudiante-b' on GitHub by visiting:
remote:   https://github.com/CristoferASG/practica-git-suntaxi/pull/new/estudiante-b
remote:
To https://github.com/CristoferASG/practica-git-suntaxi.git
 * [new branch]      estudiante-b -> estudiante-b
branch 'estudiante-b' set up to track 'origin/estudiante-b'.

C:\Users\Usuario\Documents\5to\Devops\Conexion>
```



Paso 3 y 4 : Commits realizados por ambos integrantes y Publicación de cambios al repositorio remoto

```
C:\Users\Usuario\Documents\5to\Devops\Conexion>git checkout estudiante-a
M      .dockerignore
M      .gitignore
M      app.py
Already on 'estudiante-a'
Your branch is up to date with 'origin/estudiante-a'.

C:\Users\Usuario\Documents\5to\Devops\Conexion>git add .
warning: in the working copy of '.dockerignore', LF will be replaced by CRLF the next time
Git touches it
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git
touches it

C:\Users\Usuario\Documents\5to\Devops\Conexion>git add .

C:\Users\Usuario\Documents\5to\Devops\Conexion>git commit -m "Agrega estudiante A"
[estudiante-a 39937a2] Agrega estudiante A
3 files changed, 2 insertions(+), 4 deletions(-)

C:\Users\Usuario\Documents\5to\Devops\Conexion>git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 447 bytes | 447.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/CristoferASG/practica-git-suntaxi.git
b2b0046..39937a2  estudiante-a -> estudiante-a
```

```
C:\Users\Usuario\Documents\5to\Devops\Conexion>git checkout estudiante-b
Switched to branch 'estudiante-b'
Your branch is up to date with 'origin/estudiante-b'.

C:\Users\Usuario\Documents\5to\Devops\Conexion>git add .

C:\Users\Usuario\Documents\5to\Devops\Conexion>git commit -m "Cambio del integrante B"
[estudiante-b 132e0d6] Cambio del integrante B
1 file changed, 1 insertion(+), 1 deletion(-)

C:\Users\Usuario\Documents\5to\Devops\Conexion>git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 322 bytes | 322.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/CristoferASG/practica-git-suntaxi.git
b2b0046..132e0d6  estudiante-b -> estudiante-b
```

Paso 5: Creación de Pull Requests

The screenshot shows a GitHub Pull Request (PR) page for the repository 'CristoferASG/practica-git-suntaxi'. The PR is titled 'Agrega estudiante A #1' and is currently in the 'Ready to merge' state. The PR was created by 'CristoferASG' and wants to merge 1 commit into the 'main' branch from the 'estudiante-a' branch. The PR has 0 conversations, 1 commit, 0 checks, and 3 files changed. A comment from 'CristoferASG' is visible, stating 'No description provided.' Below the comment, a green box indicates 'No conflicts with base branch' and 'Merging can be performed automatically.' A 'Merge pull request' button is present. The right sidebar shows the 'Reviews' section with 'No reviews' and a 'Convert to draft' link. The 'Assignees' section shows 'No one' and a link to 'assign yourself'. The 'Labels' section shows 'None yet'. The 'Projects' section shows 'None yet'. The 'Milestone' section shows 'No milestone'. The 'Development' section shows a link to 'View command line instructions'.

https://github.com/CristoferASG/practica-git-suntaxi/pull/1

Agrega estudiante A #1

Ready to merge Code

Open CristoferASG wants to merge 1 commit into main from estudiante-a

Conversation 0 Commits 1 Checks 0 Files changed 3

CristoferASG commented now

No description provided.

Agrega estudiante A 39937a2

No conflicts with base branch
Merging can be performed automatically.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

Add a comment

Reviews

No reviews

Still in progress? [Convert to draft](#)

Assignees

No one [assign yourself](#)

Labels

None yet

Projects

None yet

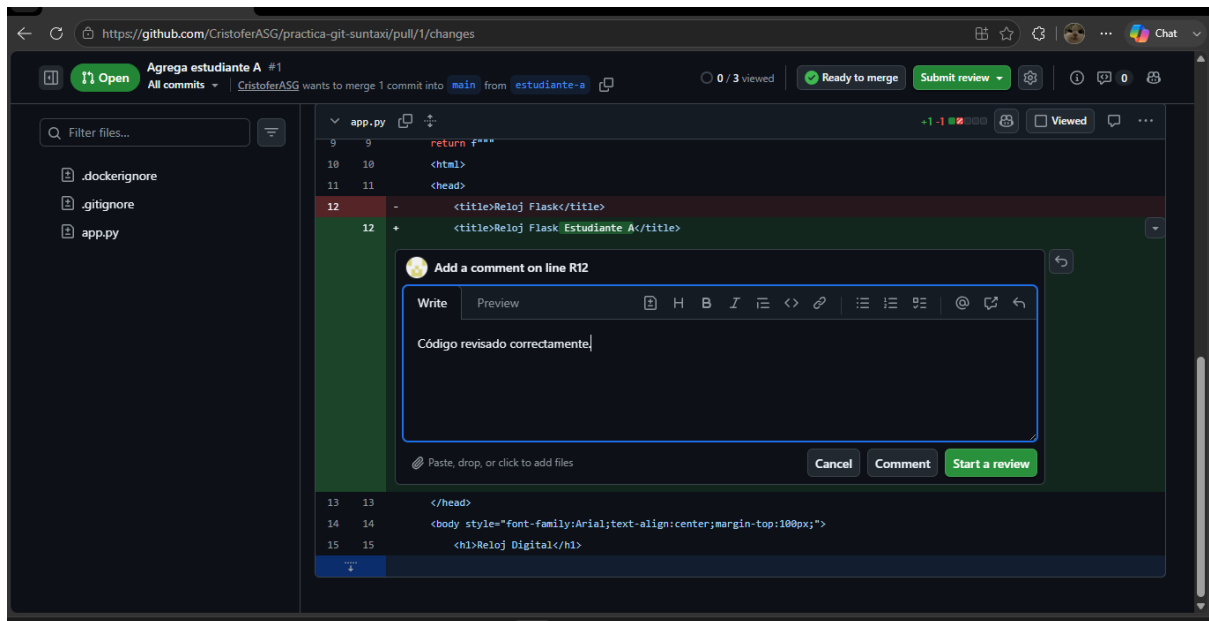
Milestone

No milestone

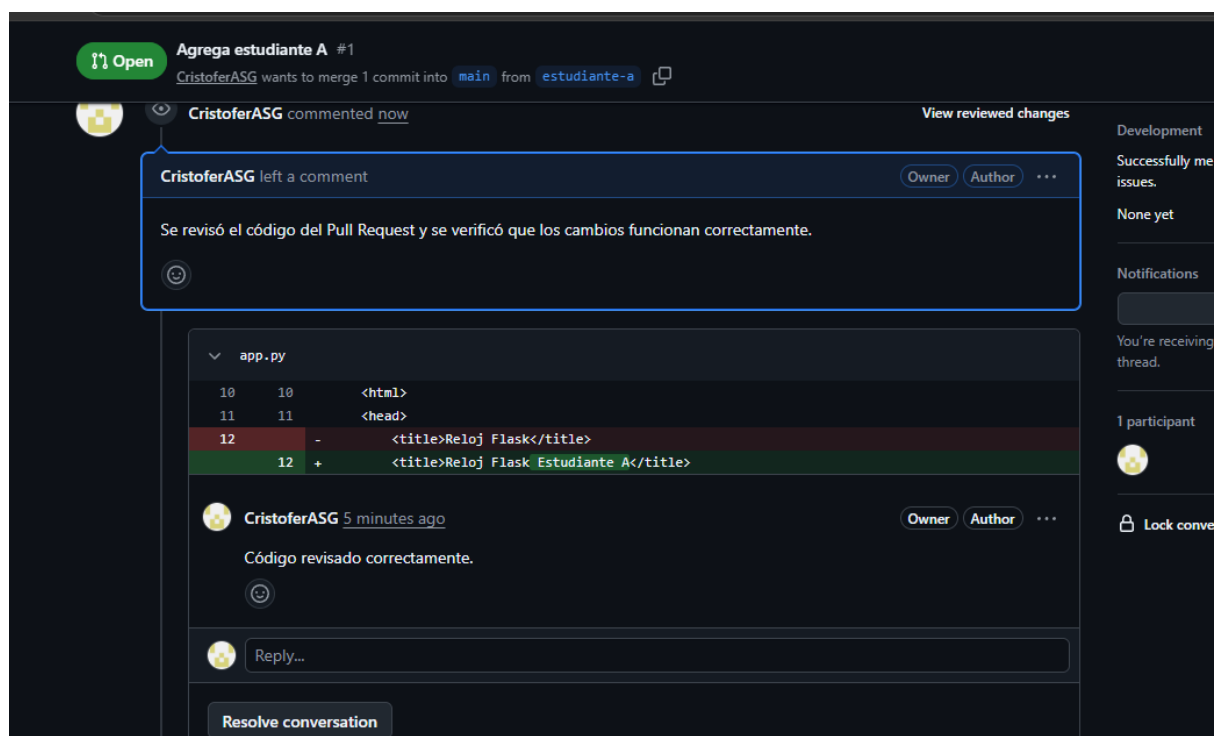
Development

[View command line instructions](#)

Paso 6: Revisión de código



Paso 7: Aprobación de Pull Requests



Paso 8: Ejecución del proceso de Merge

The screenshot displays a GitHub pull request interface. The main content area shows the pull request titled "Agrega estudiante A #1" with a green "Open" status. It indicates that "CristoferASG" wants to merge 1 commit into the "main" branch from the "estudiante-a" branch. Below the title, there are tabs for "Conversation" (3), "Commits" (1), "Checks" (0), and "Files changed" (3). The conversation section shows two comments from "CristoferASG": one from 10 minutes ago stating "No description provided." and another from 5 minutes ago stating "Código revisado correctamente." The right sidebar shows the "Merge status" section with a green checkmark and the text "No conflicts with base branch" and "Merging can be performed automatically." Below this, there is a "Merge pull request" button and links for "View command line instructions", "Convert to draft", and "Close pull request". At the bottom of the page, a notification banner states "Pull request successfully merged and closed" and "You're all set — the estudiante-a branch can be safely deleted." with a "Delete branch" button.

https://github.com/CristoferASG/practica-git-suntaxi/pull/1

CristoferASG / practica-git-suntaxi

Code Issues Pull requests 1 Agents Actions Projects Wiki

Agrega estudiante A #1

Open CristoferASG wants to merge 1 commit into main from estudiante-a

Conversation 3 Commits 1 Checks 0 Files changed 3

CristoferASG commented 10 minutes ago

No description provided.

Agrega estudiante A

CristoferASG commented 5 minutes ago

Código revisado correctamente.

Merge status Preview

✓ No conflicts with base branch
Merging can be performed automatically.

Merge pull request

You can also merge this with the command line. [View command line instructions.](#)
Still in progress? [Convert to draft](#)
Not planning to merge? [Close pull request](#)

Pull request successfully merged and closed
You're all set — the `estudiante-a` branch can be safely deleted. [Delete branch](#)

Paso 9: Aprobar Pull Request

```
C:\Users\Usuario\Documents\5to\Devops\Conexion>git push -u origin conflict-to-b
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 588 bytes | 588.00 KiB/s, done.
Total 6 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 2 local objects.
remote:
remote: Create a pull request for 'conflict-to-b' on GitHub by visiting:
remote:   https://github.com/CristoferASG/practica-git-suntaxi/pull/new/conflict-to-b
remote:
To https://github.com/CristoferASG/practica-git-suntaxi.git
 * [new branch]      conflict-to-b -> conflict-to-b
branch 'conflict-to-b' set up to track 'origin/conflict-to-b'.

C:\Users\Usuario\Documents\5to\Devops\Conexion>git checkout main
Switched to branch 'main'
Your branch and 'origin/main' have diverged,
and have 1 and 2 different commits each, respectively.
(use "git pull" if you want to integrate the remote branch with yours)

C:\Users\Usuario\Documents\5to\Devops\Conexion>git merge conflict-to-a
Already up to date.

C:\Users\Usuario\Documents\5to\Devops\Conexion>git merge conflict-to-b
Updating 884e065..67ac37f
Fast-forward
 app.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Paso 10: Generación y resolución de conflictos de integración

Acceptar cambio actual | Aceptar cambio entrante | Aceptar ambos cambios | Comparar cambios

```
15 <<<<<< Updated upstream (Cambio actual)
16 <h1>Reloj Flask</h1>
17 =====
18 <h1>Reloj Digital Estudiante B</h1>
19 >>>>>> Stashed changes (Cambio entrante)
20 <h2>{hora}</h2>
21 <p>Aplicación Flask para Dr
22 </body>
23 </html>
```

Resolver en el Editor de combinación

PROBLEMAS SALIDA TERMINAL

app.py: needs merge
error: you need to resolve your current index first

C:\Users\Usuario\Documents\Sto\Devops\Conexion>git push -u origin conflicto-a
branch 'conflicto-a' set up to track 'origin/conflicto-a'.
Everything up-to-date

C:\Users\Usuario\Documents\Sto\Devops\Conexion>git add .

C:\Users\Usuario\Documents\Sto\Devops\Conexion>git push -u origin conflicto-b
branch 'conflicto-b' set up to track 'origin/conflicto-b'.
Everything up-to-date

C:\Users\Usuario\Documents\Sto\Devops\Conexion>

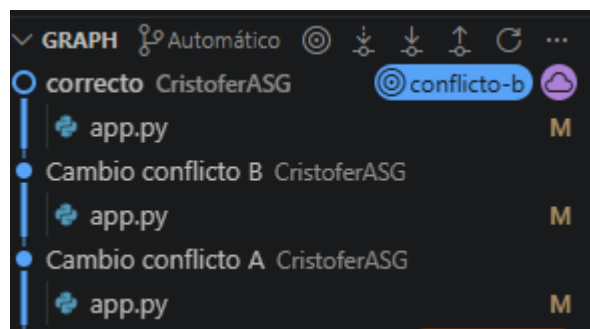
Sugerencia: Pruebe el [Plan agent](#) para investigar y planificar antes de implementar cambios.

Hay conflictos de fusión mediante combinación al aplicar el stash. Resuélvalos antes de confirmar los cambios.

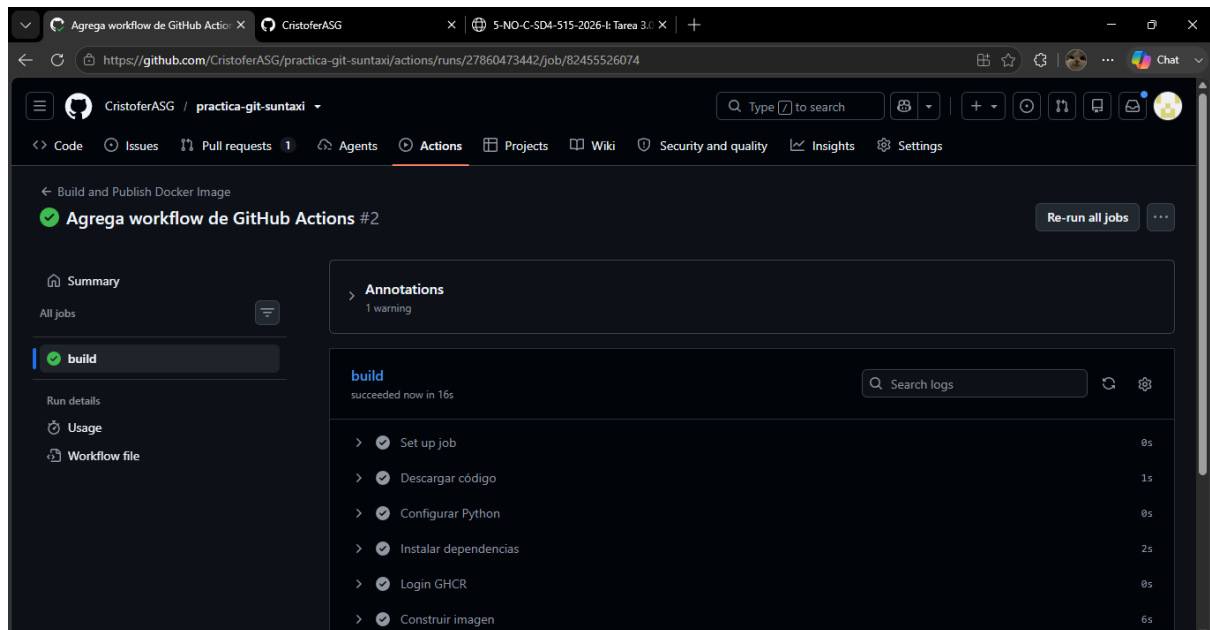
Origen: GIT.

Mostrar los cambios

Lín. 24, col. 1 Espacios: 4 UTF-8 CRLF Python Python 3.11 Prettier



Paso 10: Ejecución exitosa del workflow de GitHub Actions.



3. Información Adicional

Datos del Estudiante

Nombre completo	Cristofer Suntaxi
Carrera	Desarrollo de Software
Paralelo	5to C
URL del repositorio	https://github.com/CristoferASG/practica-git-suntaxi

Conclusiones

Realizar esta práctica de manera individual me permitió comprender a fondo ambas perspectivas del trabajo colaborativo: la del creador del repositorio y la del colaborador. Pude evidenciar la importancia de los Pull Requests como medida de control de calidad antes de integrar código a la rama principal. Asimismo, la resolución de conflictos me demostró cómo Git maneja las diferencias de código y la necesidad de comunicación en un equipo real para evitar sobrescribir el trabajo ajeno. Finalmente, la implementación de GitHub Actions me introdujo al concepto de Integración Continua, comprendiendo cómo se automatizan tareas

que en un entorno profesional ahorran tiempo, garantizan la calidad del código y previenen errores antes de llegar a producción

Bibliografía

- Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. Disponible en: <https://git-scm.com/book/en/v2>
- GitHub Docs. (2024). Documentación oficial de GitHub. Disponible en: <https://docs.github.com/es>
- Fowler, M. (2006). Continuous Integration. Martin Fowler. Disponible en: <https://martinfowler.com/articles/continuousIntegration.html>
- Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley.
- Git Documentation. (2024). Reference Manual. Disponible en: <https://git-scm.com/docs>
- GitHub Actions Documentation. (2024). Disponible en: <https://docs.github.com/en/actions>